

Trabalho Prático 2

Modelagem de Redes Sociais Acadêmicas

Data de Entrega: 25/05/2026

1 Descrição do problema

A plataforma *ConectaDCC* deseja melhorar a qualidade de suas funcionalidades de rede social acadêmica por meio do desenvolvimento de algoritmos eficientes para modelagem de conexões entre usuários. Para isso, deseja-se manter um conjunto de temas, um conjunto de usuários, as relações de interesse entre usuários e temas, e as conexões dinâmicas entre usuários ao longo do tempo.

O sistema é modelado por dois grafos distintos. O primeiro é um grafo não direcionado, não ponderado e bipartido, chamado *grafo de temas*, cujos vértices são particionados em dois conjuntos: usuários e temas. Nesse grafo, existe uma aresta entre um usuário u e um tema t se, e somente se, o tema t pertence à lista de temas de interesse do usuário u .

O segundo é um grafo direcionado e não ponderado, chamado *grafo social*, cujos vértices representam usuários. Se o usuário u segue o usuário v , então existe uma aresta direcionada de u para v . Assim, para um mesmo par de usuários, podem coexistir as duas arestas opostas $u \rightarrow v$ e $v \rightarrow u$. Nesse caso, diz-se que os dois usuários são amigos, pois cada um segue o outro.

Cada usuário possui um identificador único e seu conjunto de temas é definido no momento de seu cadastro. A partir desse cadastro, o sistema deve criar automaticamente, no grafo de temas, as arestas entre o usuário e todos os temas informados.

Além disso, o sistema deve permitir que a representação interna dos grafos seja alterada ao longo da execução. Em alguns momentos, pode ser mais interessante armazenar os grafos por meio de listas de adjacência; em outros, pode ser mais conveniente utilizar matriz de adjacência. Para refletir esse cenário, considere que o administrador da plataforma pode solicitar, durante a execução do sistema, a mudança da forma de armazenamento dos grafos, sem alterar o estado lógico das informações já cadastradas.

O sistema deve responder de forma eficiente a consultas que permitam, para um dado usuário:

1. listar seus temas;
2. listar seus seguidores;
3. listar os usuários que ele segue;
4. listar seus amigos;
5. consultar a relação de seguimento entre ele e um outro usuário, verificando se não há seguimento, se ele segue o outro, se é seguido pelo outro, ou se a relação é mútua.
6. consultar se um tema é de interesse de um usuário.
7. listar a popularidade de um tema, ou seja, indicar quantos usuários tem interesse nele.

1.1 Entrada e saída

O programa recebe como entrada um arquivo com 12 tipos de linha. Toda linha de entrada produz exatamente uma linha de saída, sempre iniciada pelo mesmo comando da entrada.

Armazenamento:

A <modo>

em que <modo> pertence a {L, M}. O modo L indica listas de adjacência, e o modo M, matriz de adjacência. A saída é igual à entrada:

A <modo>

Tema:

T <nome_tema> <tipo_tema>

Cadastra um novo tema. O campo <nome_tema> contém apenas letras e não possui espaços. O campo <tipo_tema> pertence ao conjunto {A, P}, em que A indica tema acadêmico e P indica tema pessoal. Os temas recebem identificadores inteiros não negativos únicos, em ordem crescente a partir de 0, considerando apenas os temas já cadastrados. A saída é:

T <id_tema>

Usuário:

U <nome_usuario> <idade_usuario> <id_tema_1> ...<id_tema_k>

Cadastra um novo usuário com os temas informados e insere, no grafo de temas, as arestas entre esse usuário e seus temas. O campo <nome_usuario> contém apenas letras e não possui espaços. O campo <idade_usuario> é um inteiro não negativo. Os usuários recebem identificadores inteiros não negativos únicos, em ordem crescente a partir de 0, considerando apenas os usuários já cadastrados. A saída é:

U <id_usuario>

Seguir:

S <id_1> <id_2>

Insere a aresta direcionada de <id_1> para <id_2> no grafo social. A saída é:

S <nome_usuario_1> <nome_usuario_2>

em que <nome_usuario_1> é o nome do usuário <id_1> e <nome_usuario_2> é o nome do usuário <id_2>.

Remoção de seguimento:

R <id_1> <id_2>

Remove a aresta direcionada de <id_1> para <id_2> do grafo social. A saída é:

R <nome_usuario_1> <nome_usuario_2>

em que <nome_usuario_1> é o nome do usuário <id_1> e <nome_usuario_2> é o nome do usuário <id_2>.

Consulta de temas:

LT <id_usuario>

Lista os temas associados ao usuário. A saída é:

LT <nome_usuario> <nome_tema_1> ...<nome_tema_k>

Os temas devem aparecer em ordem crescente de identificador. Se não houver temas, a saída contém apenas LT <nome_usuario>.

Consulta de seguidores:

LC <id_usuario>

Lista os usuários que seguem <id_usuario>. A saída é:

LC <nome_usuario> <nome_seguidor_1> ...<nome_seguidor_k>

Os seguidores devem aparecer em ordem crescente de identificador. Se não houver seguidores, a saída contém apenas LC <nome_usuario>.

Consulta de seguidos:

LS <id_usuario>

Lista os usuários seguidos por <id_usuario>. A saída é:

LS <nome_usuario> <nome_seguido_1> ...<nome_seguido_k>

Os seguidos devem aparecer em ordem crescente de identificador. Se não houver seguidos, a saída contém apenas LS <nome_usuario>.

Consulta de amigos:

LA <id_usuario>

Lista os amigos de <id_usuario>, isto é, os usuários que o seguem e também são seguidos por ele. A saída é:

LA <nome_usuario> <nome_amigo_1> ...<nome_amigo_k>

Os amigos devem aparecer em ordem crescente de identificador. Se não houver amigos, a saída contém apenas LA <nome_usuario>.

Consulta de relação:

Q <id_1> <id_2>

Consulta a relação de seguimento entre dois usuários. A saída é:

Q <nome_usuario_1> <nome_usuario_2> <valor>

em que <valor> é 0 se não se seguem, 1 se <id_1> segue <id_2>, 2 se <id_2> segue <id_1>, e 3 se a relação é mútua.

Consulta de interesse:

G <id_usuario> <id_tema>

Verifica se o usuário possui interesse no tema. A saída é:

G <nome_usuario> <nome_tema> <valor>

em que <valor> é 1 se o usuário possui interesse no tema, e 0 caso contrário.

Consulta de popularidade:

F <id_tema>

Consulta a popularidade do tema, definida como a quantidade de usuários interessados nele. A saída é:

F <nome_tema> <pop>

A primeira linha da entrada é sempre do tipo A. Em seguida, podem aparecer linhas dos tipos A, T, U, S, R, LT, LC, LS, LA, Q, G e F, em qualquer ordem. Todas as linhas são processadas sequencialmente, sobre o estado atual do sistema.

Os campos de nome para usuários e temas possuem apenas letras, e não possuem espaços.

1.2 Exemplo de entrada e saída

```
A L
T IA A
U Ana 20 0
T Corrida P
U Bruno 22 0 1
S 0 1
S 1 0
LT 0
LC 0
LS 0
LA 0
Q 0 1
G 1 0
F 1
A M
R 1 0
Q 0 1
```

Figura 1: Exemplo de entrada

```
A L
T 0
U 0
T 1
U 1
S Ana Bruno
S Bruno Ana
LT Ana IA
LC Ana Bruno
LS Ana Bruno
LA Ana Bruno
Q Ana Bruno 3
G Bruno IA 1
F Corrida 1
A M
R Bruno Ana
Q Ana Bruno 1
```

Figura 2: Exemplo de saída

A entrada é sempre válida: **todos os comandos respeitam o estado atual do sistema; não são referenciados usuários ou temas inexistentes, não são removidas arestas inexistentes, não são inseridas relações já existentes e toda operação que depende de cadastros anteriores utiliza apenas identificadores já cadastrados.**

Você deve implementar um programa que mantenha dinamicamente os dois grafos descritos e responda às consultas especificadas. Para isso, é obrigatória a definição, implementação e validação dos seguintes TADs:

- **Usuário:** armazena as informações associadas a um usuário, incluindo seu identificador, nome e idade.
- **Tema:** armazena as informações associadas a um tema, incluindo seu identificador, nome e tipo.
- **Grafo:** TAD suficientemente geral para modelar grafos direcionados e não direcionados, além de suportar representação por listas de adjacência e por matriz de adjacência. O TAD deve permitir a troca da representação interna durante a execução, preservando o estado lógico do grafo, isto é, sem perda de vértices ou arestas.

Esse TAD deve ser reutilizado para representar os dois grafos do problema: o grafo bipartido entre usuários e temas e o grafo social entre usuários.

Para permitir que o mesmo TAD **Grafo** represente vértices de naturezas diferentes, recomenda-se a definição de um TAD **Nó**, contendo ao menos um identificador e um tipo. O identificador indica qual usuário ou tema aquele nó representa, enquanto o tipo indica se o nó corresponde a um usuário ou a um tema. Essas duas informações devem permitir recuperar, por meio do TAD **Dicionário**, o TAD **Usuário** ou o TAD **Tema** correspondente.

- **Dicionário:** estrutura responsável por localizar os elementos do sistema a partir de seus identificadores. Esse TAD deve permitir:

- recuperar um TAD **Usuário** a partir do identificador de um usuário;
- recuperar um TAD **Tema** a partir do identificador de um tema;
- recuperar o nó de um usuário no grafo social a partir de seu identificador;
- recuperar o nó de um usuário no grafo de temas a partir de seu identificador;
- recuperar o nó de um tema no grafo de temas a partir de seu identificador.

Em outras palavras, o TAD **Dicionário** deve funcionar como uma estrutura de apoio que conecta os identificadores usados na entrada aos objetos armazenados no programa e aos nós utilizados nos grafos.

Como os identificadores de usuários e temas são contíguos e começam em 0, os mapeamentos internos desse TAD podem ser implementados de forma simples por meio de arrays indexados diretamente pelos identificadores.

Faz parte do trabalho a definição das funcionalidades desses TADs, bem como sua integração correta para manter os grafos e responder às consultas especificadas.

1.3 Fluxo de execução sugerido

Algorithm 1 Fluxo de execução sugerido

```
1: inicializar o Dicionário, o grafo social e o grafo de temas
2:  $proxIdUsuario \leftarrow 0$ ,  $proxIdTema \leftarrow 0$ 
3: for cada linha da entrada do
4:   if A <modo> then
5:     migrar os dois grafos para a representação <modo> e imprimir A <modo>
6:   else if T <nome> <tipo> then
7:      $idTema \leftarrow proxIdTema$ 
8:     criar o Tema e seu nó no grafo de temas
9:     registrar o tema e o nó no Dicionário
10:    imprimir T <idTema> e incrementar  $proxIdTema$ 
11:   else if U <nome> <idade> <temas> then
12:      $idUsuario \leftarrow proxIdUsuario$ 
13:     criar o Usuário e seus nós no grafo social e no grafo de temas
14:     registrar o usuário e seus nós no Dicionário
15:     inserir, no grafo de temas, as arestas entre o usuário e seus temas
16:     imprimir U <idUsuario> e incrementar  $proxIdUsuario$ 
17:   else if S <u> <v> then
18:     recuperar os nomes e os nós de  $u$  e  $v$  no grafo social usando o Dicionário
19:     inserir a aresta  $u \rightarrow v$  e imprimir S <nome_u> <nome_v>
20:   else if R <u> <v> then
21:     recuperar os nomes e os nós de  $u$  e  $v$  no grafo social usando o Dicionário
22:     remover a aresta  $u \rightarrow v$  e imprimir R <nome_u> <nome_v>
23:   else
24:     recuperar, no Dicionário, os usuários, temas ou nós necessários
25:     executar a consulta no grafo adequado e imprimir a saída especificada
26:   end if
27: end for
```

2 Análise Experimental

A análise experimental tem por objetivo avaliar o desempenho da solução implementada em diferentes cenários de uso da plataforma *ConectaDCC*. Em particular, deseja-se investigar como o custo das operações varia em função do tamanho da rede, da quantidade de temas por usuário, da densidade das conexões sociais e da frequência relativa entre atualizações e consultas. Para isso, sugerimos os seguintes passos:

1. Compare experimentalmente, pelo menos, dois perfis de carga:
 - **Perfil com predominância de atualizações:** a entrada contém alta frequência de operações de atualização: T, U, S, R e A, em comparação com a quantidade de consultas;
 - **Perfil com predominância de consultas:** a entrada contém alta frequência de operações de consulta: LT, LC, LS, LA, Q, G e F em comparação com a quantidade de atualizações.
2. Projete o experimento variando parâmetros que afetam diretamente o custo dos algoritmos. Entre eles, destacam-se:
 - o número total de usuários n ;
 - o número total de temas m ;
 - o número médio de temas por usuário;
 - o número total de relações de seguimento no grafo social;
 - a densidade do grafo social;
 - a frequência das atualizações;
 - a frequência das consultas;
3. Execute os experimentos e avalie os resultados, indicando claramente os parâmetros escolhidos em cada configuração experimental. Sempre que possível, reporte medidas como:
 - tempo médio de cada operação e consulta para cada modo de armazenamento;
 - uso total de memória para cada modo de armazenamento.

Em termos de implementação, o ideal é que essas medidas sejam parte do TAD e apenas informadas ao programa usuário do TAD quando pertinente. Isso vai demandar não apenas a instrumentação do código do TAD para coletar essas medidas como o armazenamento dos valores internamente no TAD. A interface do TAD deve também prover mecanismos de recuperar esses valores para fins da experimentação. Note que o uso de memória pode ser uma função do grafo instanciado, enquanto o custo deve ser mantido por função executada e/ou operação definida no arquivo de entrada.

A análise experimental deve ser conduzida com cuidado, para que as medidas obtidas reflitam de fato o comportamento da implementação avaliada. Em particular, é importante evitar que diferenças de organização do código ou de gerenciamento de memória interfiram nos resultados, produzindo tempos que não estejam relacionados diretamente ao custo dos algoritmos.

Como exemplo, suponha que a análise seja feita sobre vetores de estruturas, como um vetor de usuários ou um vetor de temas, em que cada elemento armazena diretamente seu identificador e seus atributos principais. Nesse caso, tais informações devem permanecer armazenadas de forma contígua na memória, evitando alocações dinâmicas desnecessárias e reduzindo interferências causadas por acessos indiretos. Caso contrário, o desempenho medido pode ser influenciado por custos adicionais de acesso à memória, comprometendo a validade da análise experimental.

Os itens acima constituem apenas **sugestões** para a condução da análise experimental. Os alunos têm liberdade para definir outros cenários, métricas e experimentos que considerem relevantes. No entanto, as escolhas realizadas devem ser **justificadas** e devem ser coerentes com o objetivo de avaliar o desempenho da solução proposta.

3 Pontos Extras

- Estenda o TAD **Grafo** para suportar uma terceira representação interna, preferencialmente adequada para grafos esparsos. A operação

A <modo>

deve ser estendida para aceitar um novo modo, definido e documentado, preservando a migração sem perda de vértices ou arestas.

A implementação deve ser descrita na documentação e avaliada na análise experimental. Não haverá VPL para essa funcionalidade, e o novo modo não aparecerá nos casos de teste oficiais. Contudo, a nova representação deve funcionar e pode estar sujeita a correção manual.

- Modifique seu programa para aceitar também uma nova consulta de recomendação de usuários, com o seguinte formato:

P <id_usuario> <topk> <peso_prox> <peso_afin>

em que <peso_prox> e <peso_afin> são números reais maiores ou iguais a 0.

Essa consulta deve retornar até **topk** usuários recomendados para o usuário informado.

Um usuário v é considerado válido para recomendação ao usuário u se:

- $v \neq u$;
- u não segue diretamente v .

A recomendação deve considerar duas informações: a **proximidade** entre os usuários no grafo social e a **afinidade** entre seus temas.

A distância entre u e v , denotada por $dist(u, v)$, é definida como o comprimento de um caminho direcionado mínimo de u até v , isto é, o menor número de arestas em um caminho de u até v . Assim, a proximidade é definida como:

$$prox(u, v) = \begin{cases} \frac{2}{dist(u, v)}, & \text{se existe caminho direcionado de } u \text{ até } v, \\ 0, & \text{caso contrário.} \end{cases}$$

Dessa forma, **como os usuários que u segue diretamente não são válidos para recomendação**, $prox(u, v)$ sempre pertence ao intervalo $[0, 1]$. O valor máximo ocorre quando existe $u \rightarrow s \rightarrow v$ no grafo, isto é, quando $dist(u, v) = 2$.

A afinidade entre u e v é definida como a fração de temas compartilhados pelos dois usuários em relação ao total de temas distintos associados a eles:

$$afin(u, v) = \begin{cases} 0, & \text{se } |T(u)| = |T(v)| = 0, \\ \frac{|T(u) \cap T(v)|}{|T(u) \cup T(v)|}, & \text{caso contrário.} \end{cases}$$

Assim, $afin(u, v)$ sempre pertence ao intervalo $[0, 1]$: quanto mais próximos forem os conjuntos de temas dos dois usuários, maior será a afinidade entre eles.

A pontuação de recomendação é dada por:

$$score(u, v) = peso_prox \cdot prox(u, v) + peso_afin \cdot afin(u, v).$$

Os usuários v recomendados para u devem ser os topk candidatos válidos com maior valor de $score(u, v)$.

Com essa definição, um usuário pode ser bem recomendado por estar próximo de u no grafo social, mesmo tendo poucos temas em comum, ou por ter alta afinidade temática com u , mesmo não pertencendo ao seu ciclo de conexões. Os pesos permitem controlar a importância relativa desses dois critérios em cada consulta.

A saída da consulta deve conter o comando P, seguido pelo nome do usuário consultado e pelos nomes dos usuários recomendados, ordenados por pontuação decrescente:

```
P <nome_usuario> <nome_recomendado_1> ...<nome_recomendado_k>
```

Em caso de empate, o desempate deve ser feito pelo identificador do usuário, em ordem crescente.

Se o número de usuários válidos for menor que **topk**, devem ser retornados apenas os usuários existentes. Se não houver usuários válidos, a saída deve conter apenas P <nome_usuario>.

Exemplo: considere a seguinte entrada, construída apenas para ilustrar a consulta P:

```
A L
T IA A
T ED A
T GRAFOS A
T COMPILADORES A
U Ana 20 0 1
U Bruno 21 2
U Carla 22 1 2
U Davi 23 0 2
U Eva 24 0 1 3
S 0 1
S 1 2
S 2 3
P 0 3 1.0 2.0
```

A saída produzida é:

```
A L
T 0
T 1
T 2
T 3
U 0
U 1
U 2
U 3
U 4
S Ana Bruno
S Bruno Carla
S Carla Davi
P Ana Eva Carla Davi
```

Nesse exemplo, os candidatos válidos para **Ana** são **Carla**, **Davi** e **Eva**, pois **Bruno** já é seguido diretamente por **Ana**. As pontuações são:

$$\text{score}(\text{Ana}, \text{Carla}) = 1.0 \cdot 1 + 2.0 \cdot \frac{1}{3} = \frac{5}{3},$$

$$\text{score}(\text{Ana}, \text{Davi}) = 1.0 \cdot \frac{2}{3} + 2.0 \cdot \frac{1}{3} = \frac{4}{3},$$

$$\text{score}(\text{Ana}, \text{Eva}) = 1.0 \cdot 0 + 2.0 \cdot \frac{2}{3} = \frac{4}{3}.$$

Como **Eva** e **Davi** empatam na pontuação, o desempate é feito pelo identificador do usuário. Como **Eva** possui identificador 4 e **Davi** possui identificador 3, a ordem correta seria **Davi** antes de **Eva** se o empate fosse mantido.

Para evitar esse empate, usando a entrada acima, a saída correta da consulta é:

```
P Ana Carla Davi Eva
```

Um VPL específico será utilizado para avaliar essa funcionalidade. Alunos que desejarem obter os pontos extras devem implementar corretamente essa consulta e incluir, na análise experimental, avaliação de desempenho das consultas do tipo **P**, além dos demais itens.

Os programas submetidos para os dois VPLs devem ser os mesmos. O objetivo é modificar a implementação para suportar também a operação **P, sem nenhum prejuízo para as demais operações.**

Dica: para encontrar os usuários a menor distância em termos de número de arestas, a busca em largura (BFS) pode ser uma abordagem útil.

4 Como será feita a entrega

4.1 Submissão

A entrega do TP1 compreende duas submissões:

VPL TP1: Submissão do código a ser submetido até **25/05/2026, 23:59** (o sistema vai ficar aberto madrugada adentro, mais para evitar problemas transientes de infraestrutura). Detalhes sobre a submissão do código são apresentados na Seção 4.3.

Relatório TP1: Arquivo PDF contendo a documentação do TP, assim como a avaliação experimental, conforme instruções, a ser submetido até **25/05/2026, 23:59** (o sistema vai ficar aberto madrugada adentro, mais para evitar problemas transientes de infraestrutura). Detalhes sobre a submissão de relatório são apresentados na Seção 4.2.

4.2 Documentação

A documentação do trabalho deve ser entregue em formato **PDF** e também **DEVE** seguir o modelo de relatório que será postado no `minha.ufmg`. Além disso, a documentação deve conter **TODOS** os itens descritos a seguir **NA ORDEM** em que são apresentados:

1. **Capa:** Título, nome e matrícula.
2. **Introdução:** Contém a apresentação do contexto, problema e de qual solução será empregada.
3. **Método:** Descrição da implementação, detalhando as estruturas de dados, tipos abstratos de dados (ou classes) e funções (ou métodos) implementados.
4. **Análise de Complexidade:** Contém a análise da complexidade de tempo e espaço dos procedimentos implementados, formalizada pela notação assintótica.
5. **Estratégias de Robustez:** Contém a descrição, a justificativa e a implementação dos mecanismos de programação defensiva e de tolerância a falhas implementados.
6. **Análise Experimental:** Apresenta os experimentos realizados em termos de desempenho computacional¹, assim como as análises dos resultados.
7. **Conclusões:** A Conclusão deve conter uma frase inicial sobre o que foi feito no trabalho. Posteriormente deve-se sumarizar o que foi aprendido.
8. **Bibliografia:** Contém fontes utilizadas para realização do trabalho. A citação deve estar em formato científico apropriado, escolhido por você.
9. Número máximo de páginas incluindo a capa: 10

A documentação deve conter a descrição do seu trabalho em termos funcionais, com foco em algoritmos, estruturas de dados e decisões de implementação importantes durante o desenvolvimento.

Evite a descrição literal do código-fonte na documentação do trabalho.

Dica: Sua documentação deve ser clara o suficiente para que uma pessoa (da área de Computação ou não) consiga ler, entender o problema tratado e como foi feita a solução. A documentação deverá ser entregue como uma atividade separada designada para tal no `minha.ufmg`. A entrega deve ser um arquivo `.pdf`, nomeado `nome_sobrenome_matricula.pdf`, onde nome, sobrenome e matrícula devem ser substituídos por suas informações pessoais.

¹Para este trabalho, não é necessário analisar a localidade de referência.

4.3 Código

Você deve utilizar a linguagem C ou C++ para o desenvolvimento do seu sistema. O uso de estruturas pré-implementadas pelas bibliotecas-padrão da linguagem ou terceiros é terminantemente vetado. Você DEVE utilizar a estrutura de projeto abaixo junto ao Makefile:

```
– TP
  |– src
  |– bin
  |– obj
  |– include
  Makefile
```

A pasta **TP** é a raiz do projeto; **src** deve armazenar arquivos de código (*.c, *.cpp, ou *.cc); a pasta **include**, os cabeçalhos (headers) do projeto, com extensão *.h, por fim as pastas **bin** e **obj** devem estar vazias. O Makefile deve estar na raiz do projeto. A execução do Makefile deve gerar os códigos objeto *.o no diretório **obj** e o executável do TP no diretório **bin**. O arquivo executável DEVE se chamar **tp1.out** e deve estar localizado na pasta **bin**. O código será compilado com o comando:

```
make all
```

O seu código será avaliado através de uma **VPL** que será disponibilizada no moodle. Você também terá à disposição uma VPL de testes para verificar se a formatação da sua saída está de acordo com a requisitada. A VPL de testes não vale pontos nem conta como trabalho entregue. Um pdf com instruções de como enviar seu trabalho para que ele seja compilado corretamente estará disponível no Moodle.

5 Avaliação

- Corretude na execução dos casos de teste - (15% da nota total)
- Indentação, comentários do código fonte e uso de boas práticas - (10% da nota total)
- Conteúdo segundo modelo proposto na seção **Documentação**, com as seções detalhadas corretamente - (20% da nota total)
- Definição e implementação das estruturas de dados e funções - (10% da nota total)
- Apresentação da análise de complexidade das implementações - (10% da nota total)
- Análise experimental - (30% da nota total)
- Aderência completa às instruções de entrega - (5% da nota total)

Se o programa submetido **não compilar**² ou se compilar mas não passar em **pelo menos um caso de teste**, seu trabalho não será avaliado e sua nota será **0**. Trabalhos entregues com atraso sofrerão **penalização de 2^{d-1} pontos**, com d = dias úteis de atraso.

²Entende-se por compilar aquele programa que, independente de erros no Makefile ou relacionados a problemas na configuração do ambiente, funcione e atenda aos requisitos especificados neste documento em um ambiente Linux.

6 Considerações finais

1. Comece a fazer esse trabalho prático o quanto antes, enquanto o prazo de entrega está tão distante quanto jamais estará.
2. Leia atentamente o documento de especificação, pois o descumprimento de quaisquer requisitos obrigatórios aqui descritos causará penalizações na nota final.
3. Certifique-se de garantir que seu arquivo foi submetido corretamente no sistema.
4. Plágio é crime. Trabalhos onde o plágio for identificado serão **automaticamente anulados** e as medidas administrativas cabíveis serão tomadas (em relação a todos os envolvidos). Discussões sobre o trabalho entre colegas são permitidas. É permitido consultar fontes externas, desde que exclusivamente para fins didáticos e devidamente registradas na seção de bibliografia da documentação. **Cópia e compartilhamento de código não são permitidos.**
5. A utilização de ferramentas de inteligência artificial é de sua responsabilidade, ou seja, quaisquer erros, plágios ou dificuldade de entendimento associada a conteúdo generativo será devidamente penalizada.

7 FAQ (*Frequently asked Questions*)

1. Posso utilizar qualquer versão do C++? NÃO, o corretor da VPL utiliza C++11.
2. Posso fazer o trabalho no Windows, Linux, ou MacOS? SIM, porém lembre-se que a correção é feita sob o sistema Linux, então certifique-se que seu trabalho está funcional em Linux.
3. Posso utilizar alguma estrutura de dados do C++ do tipo Queue, Stack, Vector, List, etc? NÃO.
4. Posso utilizar smart pointers? NÃO.
5. Posso utilizar o tipo String? SIM.
6. Posso utilizar o tipo String para simular minhas estruturas de dados? NÃO.
7. Posso utilizar alguma biblioteca para tratar exceções? SIM.
8. Posso utilizar alguma biblioteca para gerenciar memória? SIM.
9. As análises e apresentação dos resultados são importantes na documentação? SIM.
10. Os meus princípios de programação ligados a C++ e relacionados a engenharia de software serão avaliados? NÃO.
11. Posso fazer o trabalho em dupla ou em grupo? NÃO.
12. Posso trocar informações com os colegas sobre os fundamentos teóricos do trabalho? SIM.
13. Posso utilizar IDEs, Visual Studio, Code Blocks, Visual Code, Eclipse? SIM.